Sustavi baza podataka

Predavanja

7. Sustav za obnovu

(2. dio)

travanj 2026.

Dnevnik, dnevnik izmjena, fizički dnevnik, logički dnevnik

- aktivnosti transakcija se bilježe u **dnevniku** (*log*, *journal*)
- jedan oblik implementacije dnevnika je **dnevnik izmjena** (*update log*).
  - sadrži zapise dnevnika izmjena
    - <start T_i>, <T_i, x_j, V_{old}, V_{new}>, <commit T_i>, <abort T_i>

- **novi pojmovi: fizički dnevnik, logički dnevnik**
  - nekonzistentna upotreba ovih pojmova u literaturi
  - (Silberschatz, 2010.)
    - fizički dnevnik sadrži zapise oblika *old-value*, *new-value*
    - logički dnevnik sadrži opise *operacija* koje su obavljene
      - za obavljanje kompenzacijskih operacija tijekom obnove
  - (Garcia-Molina, 2000.)
    - fizički dnevnik sadrži zapise oblika: *old-block*, *new-block*
    - logički dnevnik sadrži zapise oblika: *old-value*, *new-value*
    - u nastavku se koriste termini iz (Garcia-Molina, 2000.)

Primjer sadržaja logičkog dnevnika u sustavu IBM Informix

Informativno

osoba

oib	ime	prez
12345678901	Ivica	Horvat

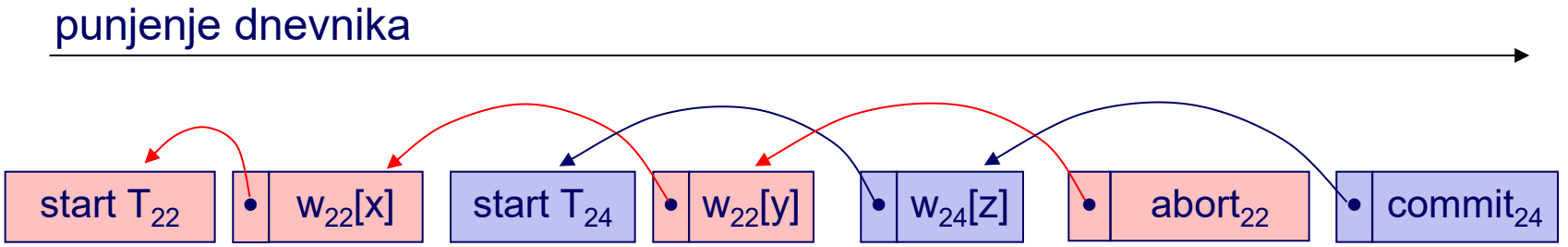
Korisnici **miha** i **jure** obavljaju naredbe:

```
(m) BEGIN WORK;
(m) UPDATE osoba SET
    ime='Marko', prez='Ban'
    WHERE oib = '12345678901';
(j) BEGIN WORK;
(j) INSERT INTO osoba
    VALUES('22233322233',
    'Mendo', 'Slavica');
(m) INSERT INTO osoba
    VALUES('3334444555',
    'Ivica', 'Horvat');
(m) COMMIT WORK;
(j) COMMIT WORK;
```

```
# onlog -l -n 3360
addr     len  type     xid      id    link
48018    52   BEGIN    22       3360          12/22/2008 19:40:17 31  miha
         34000000 200d0100 00000000 00000000 4... ... ........
         [skraćeno]                           ....
4804c    104  HUPDAT   22                     48018    200f5e   101  0   31  31  2
         68000000 00004900 12000000 00000000 h.....I. ........
         00000000 00000000 16000000 18800400 ........ ........
         7e16e100 5e0f2000 5e0f2000 01010000 ~...^. . ^. .....
         00000000 1f001f00 02000000 00000000 ........ ........
         7075626c 000b0006 49766963 61204d61 publ.... Ivica Ma
         726b6f20 00150007 486f7276 61742042 rko .... Horvat B
         616e2020 20202020                   an
480b4    52   BEGIN    24       3360          12/22/2008 19:40:29 41  jure
         34000000 200d0100 00000000 00000000 4... ... ........
         [skraćeno]
480e8    96   HINSERT  24                     480b4    200f5e   201      31
         [skraćeno]
         32323233 33333232 3233334d 656e646f 22233322 233Mendo
         20202020 20536c61 76696361 20202000      Sla vica    .
48148    96   HINSERT  22                     4804c    200f5e   102      31
         [skraćeno]
         33333334 34343435 35352049 76696361 33344445 55 Ivica
         20202020 20486f72 76617420 20202000      Hor vat     .
481a8    48   COMMIT   22                     48148    12/22/2008 19:40:36
         [skraćeno]
```

addr – *Log record address*
len – *record length*
type – *record type name*
xid – *transaction number*
id – *logical log number*
link – *link to the previous record in the transaction*

Efikasno poništavanje transakcija



- zapisi različitih transakcija su u dnevnik upisani sekvencijalno i naizmjence
- poništavanje transakcija zbog pogreške transakcije ili eksplicitnog zahtjeva za poništavanjem transakcije obavlja se izvršavanjem operacije *undo* za sve zapise izmjene te transakcije
- operacije *undo* se obavljaju u smjeru od kraja dnevnika prema početku
- sustav mora imati mogućnost brzog pronalaženja svih prethodnih zapisa određene transakcije (počevši od zapisa abort/rollback) - to omogućuje pokazivač prema prethodnom zapisu iste transakcije u dnevniku

Kontrolna točka

Problem:

- za sve do sada prikazane tehnike pretpostavljali smo da se u slučaju **pogreške sustava** obnova mora provesti na temelju **cijelog** dnevnika
- koliko je dnevnik velik?
  - ovisi o učestalosti operacija izmjena i veličini podataka koji se mijenjaju

ISVU - Informacijski sustav visokih učilišta u RH (stanje 2009. godine)

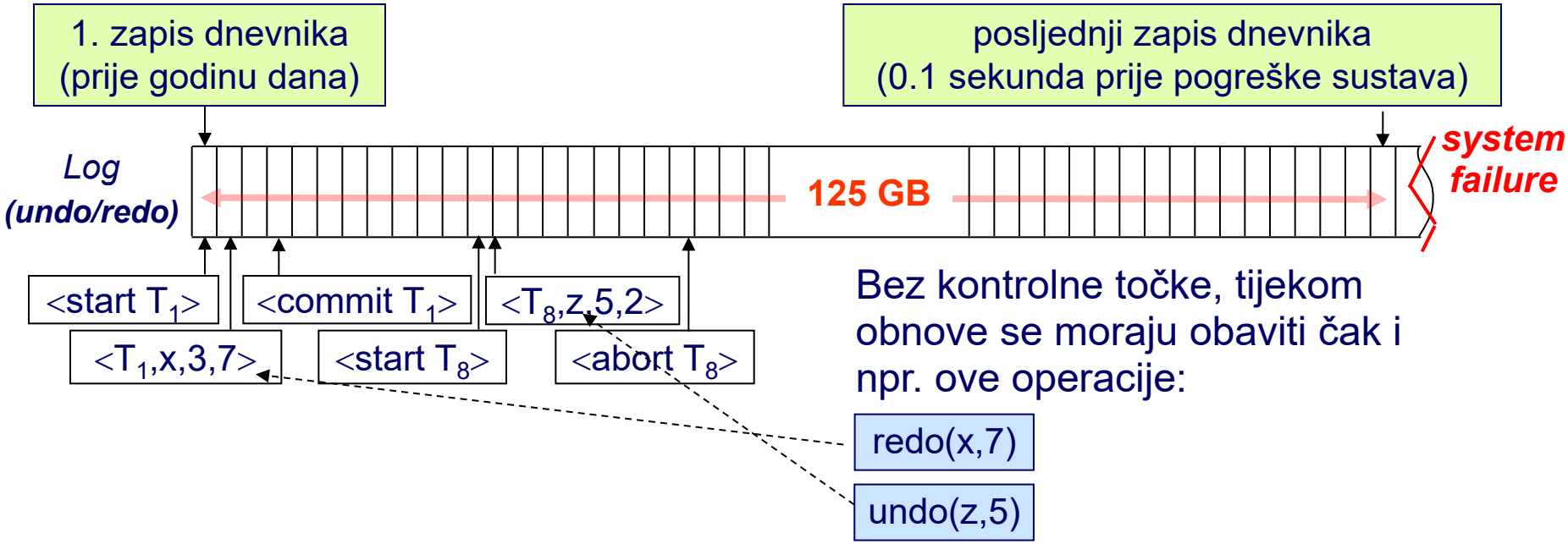
350 000 studenata

1 000 000 izdanih potvrda

5 000 000 ispita

prirast dnevnika: prosječno 350 MB/dan $\Rightarrow$ 125 GB/godinu

Kontrolna točka



Problemi koji se rješavaju upotrebom kontrolne točke

1. Trajanje obnove: satima? danima?
   - Kako bi postupak obnove korištenjem cijelog dnevnika utjecao na srednje vrijeme potrebno za oporavak (MTTR) i raspoloživost sustava (*availability*)?
2. Čuvanje cijelog dnevnika u *online* stabilnoj memoriji: godinama?

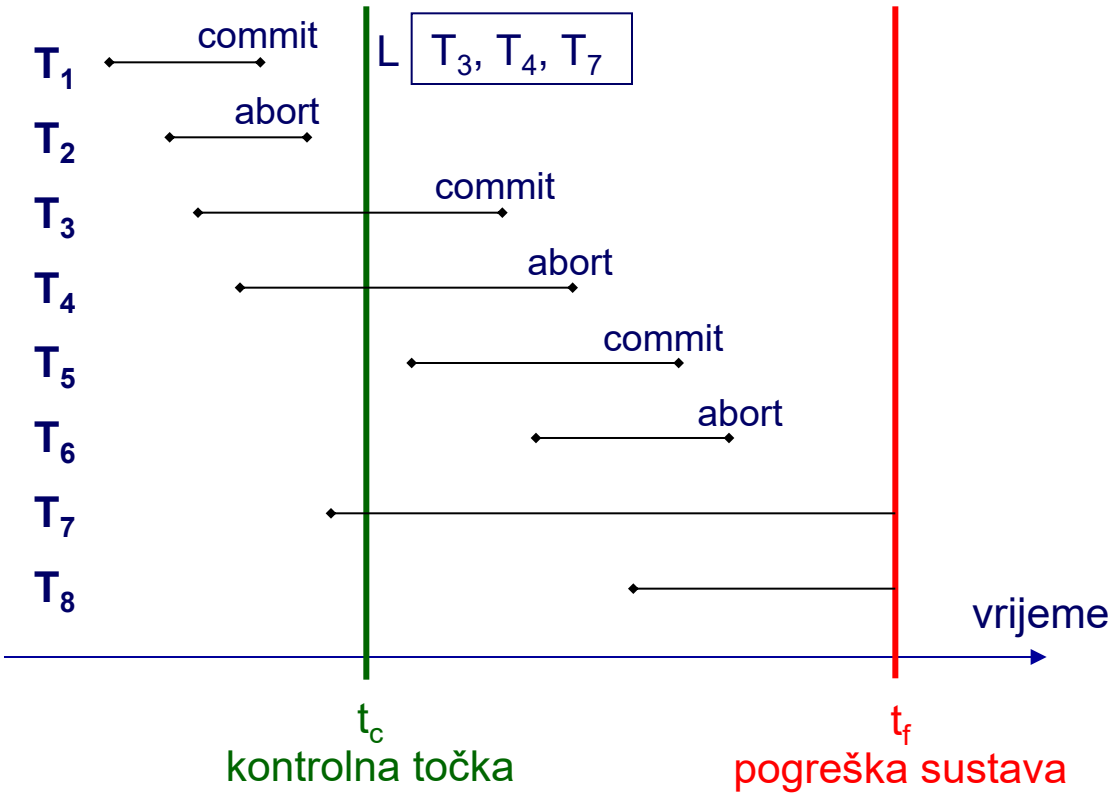
Kontrolna točka - obavljanje kontrolne točke

Postavljanje/obavljanje kontrolne točke

- periodički (npr. svakih 10 minuta) sustav određuje kontrolnu točku (*checkpoint*)
  1. SUBP prestaje obavljati operacije koje zahtijevaju pisanje u međuspremnik podataka ili međuspremnik dnevnika
  2. sadržaj međuspremnika dnevnika (*log buffer*) se zapisuje u stabilnu memoriju
  3. stranice međuspremniku podataka (*data buffer*) čiji je sadržaj promijenjen (*dirty pages*) zapisuje se u postojanu memoriju
  4. u dnevnik se upisuje zapis <chkpt L>* (+ *flush log*)
  5. SUBP nastavlja obavljati operacije koje zahtijevaju pisanje u međuspremnik podataka ili dnevnika

* L je lista identifikatora svih transakcija T_1, T_2, ..., T_k koje su bile aktivne u trenutku kontrolne točke.

Kontrolna točka - obnova nakon pogreške sustava



- T_1 i T_2 su započele i završile prije t_c. T_1 je potvrđena, T_2 je poništena
- T_3 i T_4 su započele prije t_c, završile između t_c i t_f. T_3 je potvrđena, T_4 je poništena
- T_5 i T_6 su započele i završile između t_c i t_f. T_5 je potvrđena, T_6 je poništena
- T_7 je započela prije t_c i bila je aktivna u trenutku t_f
- T_8 je započela između t_c i t_f i bila je aktivna u trenutku t_f

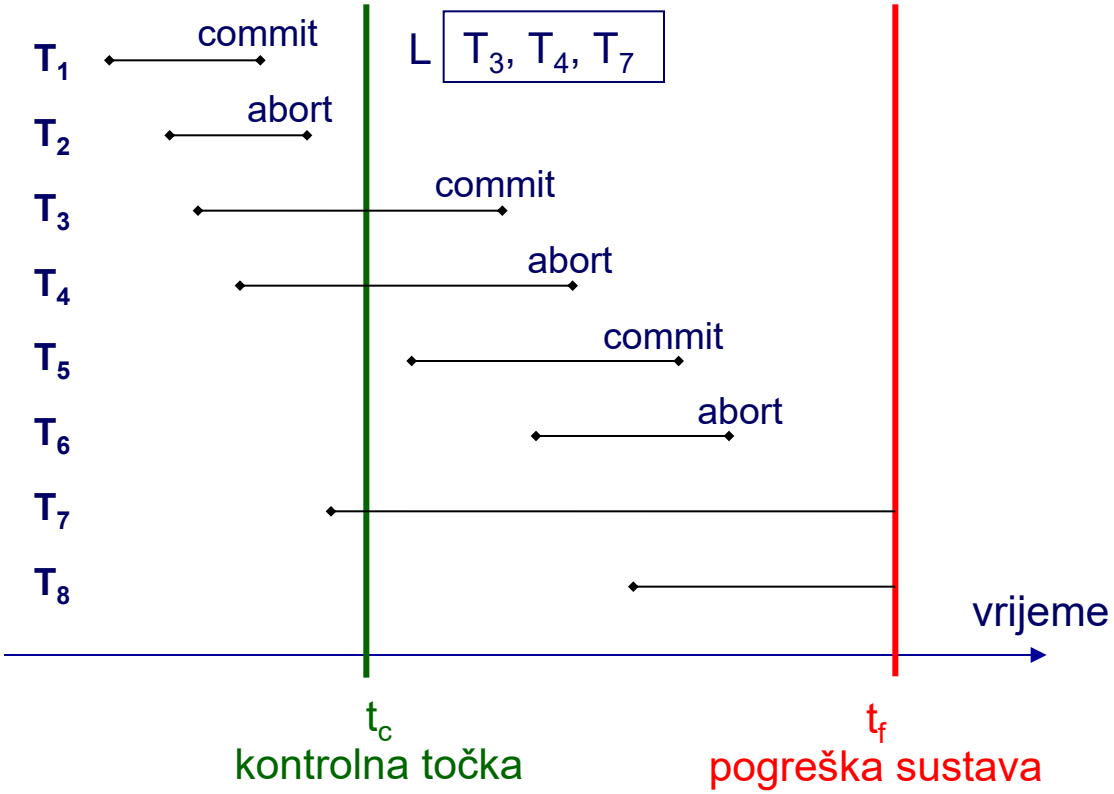
Kontrolna točka - postupak obnove nakon pogreške sustava

- formiraj dvije prazne liste: redo-list i undo-list
- čitaj dnevnik unatrag od kraja dnevnika do zadnjeg <chkpt L> zapisa
  - za svaki zapis oblika <commit T_i> dodaj T_i u redo-list
  - za svaki zapis oblika <start T_i>, ako se T_i ne nalazi u redo-list, dodaj T_i u undo-list
- svaku transakciju T_i iz L (iz <chkpt L>) koja nije u redo-list dodaj u undo-list

- čitaj dnevnik unatrag od kraja
  - za svaki zapis <T_i, x , V_1, V_2>, ako je T_i u undo-list, obavi *undo(x, V_1)*
  - čitanje dnevnika unatrag prestaje kad se pronađu zapisi <start T_i> za svaku transakciju T_i iz undo-list (neki od tih zapisa mogu se u dnevniku nalaziti i prije zapisa kontrolne točke)
- čitaj dnevnik od zadnje kontrolne točke do kraja dnevnika (*roll forward*)
  - za svaki zapis <T_i, x , V_1, V_2>, ako je T_i u redo-list, obavi *redo(x, V_2)*

Kontrolna točka - postupak obnove nakon pogreške sustava

Primjer:



undo-list	redo-list
T8 T6 T4 T7	T5 T3

- *undo* operacije će se obaviti za sve zapise dnevnika transakcija T_4, T_6, T_7 i T_8
- *redo* operacije će se obaviti za zapise dnevnika transakcija T_3 i T_5 koji su nastali nakon t_c

Kontrolna točka

O redoslijedu koraka koji se obavljaju u kontrolnoj točki

1. SUBP prestaje obavljati operacije koje zahtijevaju pisanje u međuspremnik podataka ili međuspremnik dnevnika
2. sadržaj međuspremnika dnevnika (*log buffer*) se zapisuje u stabilnu memoriju
3. stranice međuspremniku podataka (*data buffer*) čiji je sadržaj promijenjen (*dirty pages*) zapisuje se u postojanu memoriju
4. u dnevnik se upisuje zapis <chkpt L>* (+ *flush log*)
5. SUBP nastavlja obavljati operacije koje zahtijevaju pisanje u međuspremnik podataka ili dnevnika

Za vježbu odgovoriti na pitanja:

- zašto se korak 1 mora obaviti prije koraka 2?
- zašto se korak 2 mora obaviti prije koraka 3?
- zašto se korak 3 mora obaviti prije koraka 4?
- zašto se za zapise dnevnika prije kontrolne točke nikad ne mora obavljati operacija *redo*, a ponekad se mora obavljati operacija *undo*

Upravljanje logičkim dnevnicima

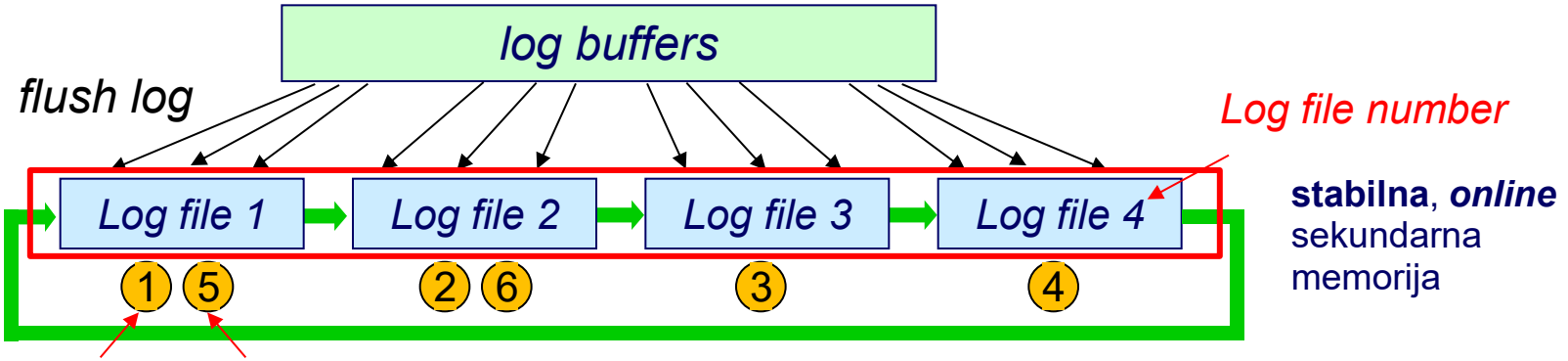
- s obzirom na mogući veliki prirast dnevnika, potreban je efikasan mehanizam upravljanja dnevnicima
- zapisi dnevnika se upisuju u (*online*) stabilnu memoriju (*raw* ili *cooked device*)
  - *online* memorija: povezana s računalom, neprekidno i odmah dostupna bez intervencije operatera
  - *offline* memorija: uglavnom se odnosi na trake, optičke medije, *pluggable* HDD, *pluggable* SSD i slično

- zapisi dnevnika se operacijom *flush log* upisuju u *online* stabilnu memoriju
  - tamo se zadržavaju najmanje onoliko koliko je to potrebno radi poništavanja transakcija i obnove nakon pogreške sustava
- a zatim se mogu obrisati (iz *online*) i prebaciti u *offline* stabilnu memoriju
  - tamo se čuvaju barem onoliko dugo koliko bi moglo biti potrebno za obnovu sustava nakon pogreške medija

Upravljanje logičkim dnevnicima

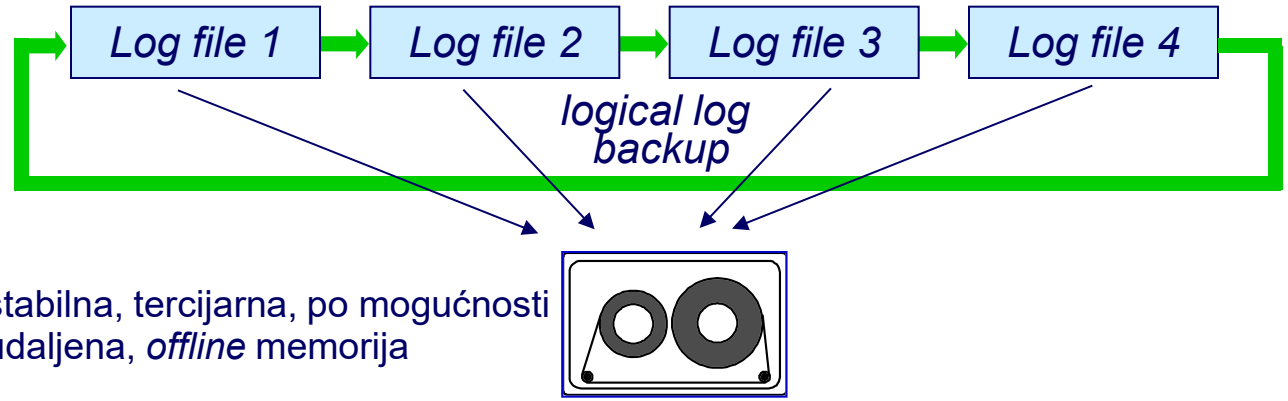
IBM Informix

- prostor za pohranu dnevnika (*logical log*) smješten je u jedan od prostora baze podataka (npr. *rootdbspace*), u stabilnoj (*online*) memoriji
- prostor za pohranu dnevnika je podijeljen u zasebne cjeline koje se nazivaju datoteke dnevnika (*logical log file*). Oracle: *redo log group*
- sustav posjeduje najmanje tri datoteke dnevnika, ali se najčešće koristi veći broj, npr. 50 ili 500
- datoteke dnevnika se ciklički izmjenjuju: kad se jedna datoteka dnevnika popuni, zapisi dnevnika se počinju upisivati u sljedeću. Kad se popuni posljednja datoteka dnevnika u nizu, ponovo se koristi prva



Upravljanje logičkim dnevnicima

- čim se popuni, sadržaj datoteke dnevnika se može arhivirati (*logical log backup*)
  - na traku, optički medij
  - na udaljeni sustav za pohranu podataka (*remote backup site*)



- stabilna, sekundarna, *online* memorija
- uvijek sadrži sve što je potrebno za poništavanje transakcija ili obnovu nakon pogreške sustava

stabilna, tercijarna, po mogućnosti udaljena, *offline* memorija

Arhiviranjem dnevnika:

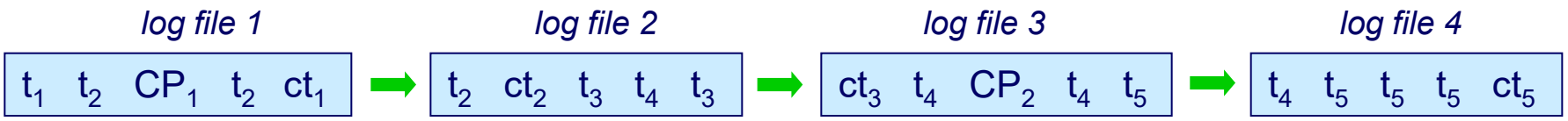
- sadržaj dnevnika se (dodatno) štiti od pogreške medija
  - jer dnevnik je vrlo važan za mogućnost obnove nakon pogreške medija (kasnije)
- omogućava se ponovno korištenje datoteke dnevnika
- može li se prostor datoteke dnevnika koja je arhivirana odmah početi puniti novim zapisima dnevnika?
  - **ne uvijek!**

Koliko dugo čuvati dnevnik u dostupnoj stabilnoj memoriji

- zapisi logičkih dnevnika koji se odnose na trenutno aktivne transakcije potrebni su za poništavanje neispravne (*failed*) transakcije (bilo zbog pogreške transakcije, bilo zbog izvršene naredbe ROLLBACK), a u postupku obnove nakon pogreške sustava nužni su svi zapisi dnevnika o transakcijama koje su bile aktivne tijekom posljednje kontrolne točke ili kasnije
- stoga se u uvijek dostupnoj (*online*), stabilnoj memoriji moraju čuvati:
  - datoteke dnevnika koje još uvijek nisu arhivirane i sve datoteke dnevnika koje slijede iza njih
  - datoteka dnevnika koja sadrži posljednju kontrolnu točku i sve datoteke dnevnika koje slijede iza nje
  - datoteke dnevnika koje sadrže zapise transakcija koje su bile aktivne u trenutku posljednje kontrolne točke (za vježbu odgovorite zašto) i sve datoteke dnevnika koje slijede iza njih

Koliko dugo čuvati dnevnik u dostupnoj stabilnoj memoriji

Primjer:



- t_i je oznaka za zapis dnevnika koji pripada transakciji T_i
- ct_i je oznaka zapisa dnevnika <commit T_i>
- CP je oznaka kontrolne točke

- uz pretpostavku da su sve prikazane datoteke dnevnika arhivirane
  - datoteka dnevnika 3. (i sve datoteke koje slijede) se mora čuvati jer datoteka dnevnika 3. sadrži posljednju kontrolnu točku
  - datoteka dnevnika 2. (i sve datoteke koje slijede) se mora čuvati jer sadrži zapise transakcije T_4 koja je bila aktivna tijekom posljednje kontrolne točke

Problem dugih transakcija

- transakcije koje relativno dugo traju ili produciraju veliki broj zapisa dnevnika mogu uzrokovati stanje u kojem na raspolaganju nema slobodnih datoteka dnevnika

  $\Rightarrow$ blokiranje sustava

  - npr. ako transakcija T_4 iz prethodnog primjera ne završi prije nego na red za punjenje dođe datoteka *log file 2*, sustav prestaje zaprimati nove transakcije

Problem dugih transakcija

IBM Informix - različiti načini sprječavanja blokiranja sustava

1. sustav se može konfigurirati tako da se u slučaju potrebe omogući automatsko ili ručno dodavanje datoteka dnevnika
   - potrebno je unaprijed predvidjeti prostor u stabilnoj memoriji

ili

2. DBA definira granične vrijednosti ukupne popunjenosti dnevnika (LTXHWM i LTXEHWM)
   - prvi znak za opasnost (*LTXHWM-Long Transaction High Water Mark*): npr. nakon što je 70% ukupne veličine dnevnika popunjeno, sustav prestaje prihvaćati nove transakcije
   - drugi znak za opasnost (*LTXEHWM-Long Transaction Exclusive access High Water Mark*): npr. nakon što je 90% ukupne veličine dnevnika popunjeno, sustav počinje poništavati sve aktivne transakcije

Kako ispravno odrediti veličinu i broj datoteka dnevnika

IBM Informix

- LOGSIZE: veličina datoteke dnevnika
- LOGFILES: broj datoteka dnevnika

⇒ ukupna veličina dnevnika = LOGSIZE×LOGFILES

- pri određivanju ukupne veličine dnevnika trebalo bi uzeti u obzir:
  - tipično vrijeme trajanja transakcija (prevladavaju li konverzacijske transakcije, *online* transakcije ili *batch* transakcije)
  - broj operacija koje transakcije obavljaju
  - vrstu operacija koje transakcije obavljaju (čitanje ili pisanje)
  - opterećenje sustava (može produljiti vrijeme odziva, odnosno trajanje transakcija)

> *It is difficult to predict how much logical-log space your database server system requires until the system is fully in use* (IBM Informix Dynamic Server Performance Guide)

- pri određivanju veličine datoteke dnevnika treba uzeti u obzir:
  - veće datoteke dnevnika pružaju nešto bolje performanse
    - vrijedi li riskirati svojstvo izdržljivosti transakcija?
  - tj. koliko transakcija smije biti izgubljeno u slučaju kvara medija (*online* stabilne memorije) na koji se pohranjuju datoteke dnevnika?
    - male datoteke dnevnika će biti brže arhivirane

Obnova nakon pogreške medija

Pogreška medija

- transakcije koje su bile u tijeku su prekinute (kao i kod pogreške sustava)
- izgubljen je sadržaj baze podataka u postojanoj memoriji
- ekstremni slučaj: možda je izgubljen čak i dnevnik u *online* stabilnoj memoriji

Rješenje:

- periodička izrada redundantnih kopija (arhiva) sadržaja postojane memorije
- neprekidno arhiviranje stabilne *online* memorije (dnevnik iz *online* stabilne memorije čim je prije moguće kopirati u *offline*, po mogućnosti udaljenu, stabilnu memoriju)

Obnova nakon pogreške medija

Smještaj redundantnih kopija:

- nije važan broj izrađenih kopija, nego mjesto na kojem se te kopije čuvaju!
  - kopija se moraju čuvati na mediju s nezavisnim modalitetom kvarova (*independant failure mode*). Mediji imaju nezavisan modalitet kvarova ako niti jedan zasebni događaj (predviđen u okviru razumnih pretpostavki) neće uzrokovati istovremeni kvar na oba medija

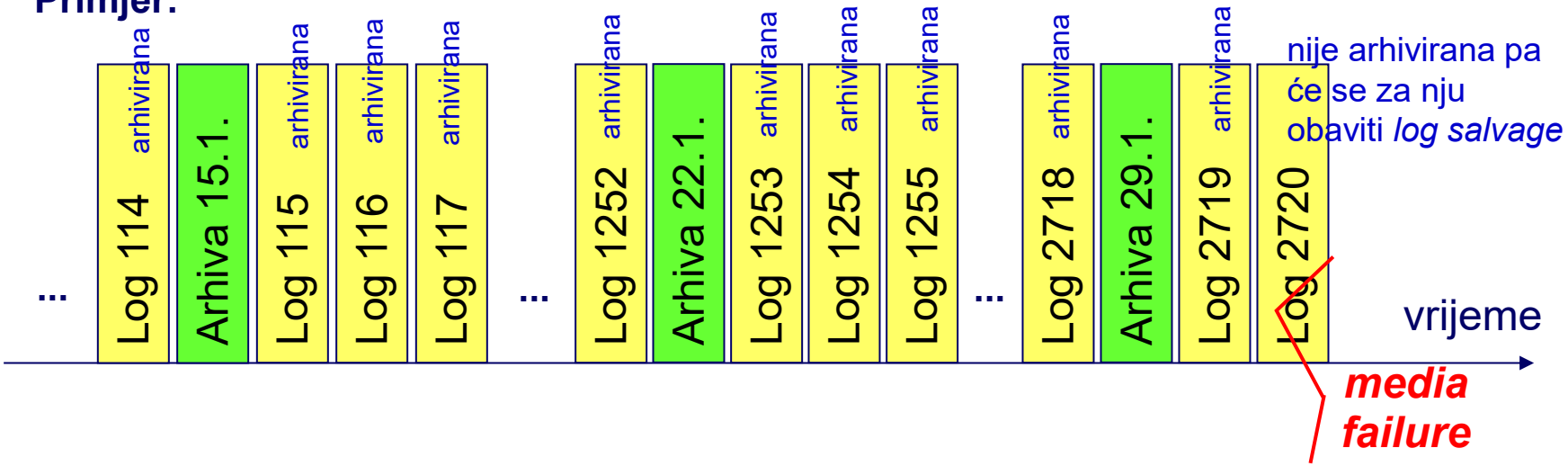
događaj	primjer uređaja s nezavisnim modalitetom kvarova
kvar magnetskog diska	drugi disk ili jedinica trake u istom računalu
požar u prostoriji	magnetski disk u računalu u susjednoj prostoriji
požar u zgradi	disk ili jedinica trake u računalu u susjednoj zgradi
katastrofalan potres	disk jedinica u drugom, dovoljno udaljenom gradu

Obnova nakon pogreške medija

- periodičko kopiranje cijelog sadržaja baze podataka (*database dump*, *database backup*) na uređaj s nezavisnim modalitetom kvarova
- u slučaju kvara medija, baza podataka se uz pomoć te kopije može vratiti u stanje u kojem je bila **u trenutku izrade posljednje arhive**
- **ali, to nije dovoljno!**
- za obnovu nakon pogreške medija potrebna je
  - posljednja arhiva i
  - svi zapisi dnevnika koje su nastali nakon početka izrade posljednje arhive
  - **što ako se neke datoteke dnevnika nisu stigle arhivirati prije pogreške medija?**
    1. slučaj kada je *online* stabilna memorija ostala sačuvana - prije obnove provesti "*log salvage*" - arhivirati sadržaj datoteka dnevnika koje se nisu stigle arhivirati prije kvara medija
    2. ekstremni slučaj kada je i *online* stabilna memorija uništena - to će izazvati sigurni gubitak nekih transakcija (odnosno nije zadovoljeno svojstvo izdržljivosti transakcija)

Arhiviranje baze podataka i datoteka dnevnika

Primjer:



- preporuča se čuvati barem tri posljednje arhive i sve pripadne datoteke dnevnika. Zašto?
  - u slučaju gubitka posljednje arhive (npr. *tape read error*), moguća je obnova uz pomoć prethodne arhive i svih dnevnika nastalih nakon te arhive
- obnova sustava je moguća pomoću:
  - arhiva 29.1. + sve datoteke dnevnika 2719 - 2720 ili
  - arhiva 22.1. + sve datoteke dnevnika 1253 - 2720 ili
  - arhiva 15.1. + sve datoteke dnevnika 115 - 2720

Inkrementalno arhiviranje baze podataka

- Problem: postupak arhiviranja *cijele* baze podataka je dugotrajan i zahtijeva puno prostora, a ako se obavlja rijetko, obnova u posljednje konzistentno stanje pomoću svih dnevnika nastalih poslije davno napravljene arhive može (pre)dugo trajati
  - rješenje: inkrementalno arhiviranje ili arhiviranje na više razina

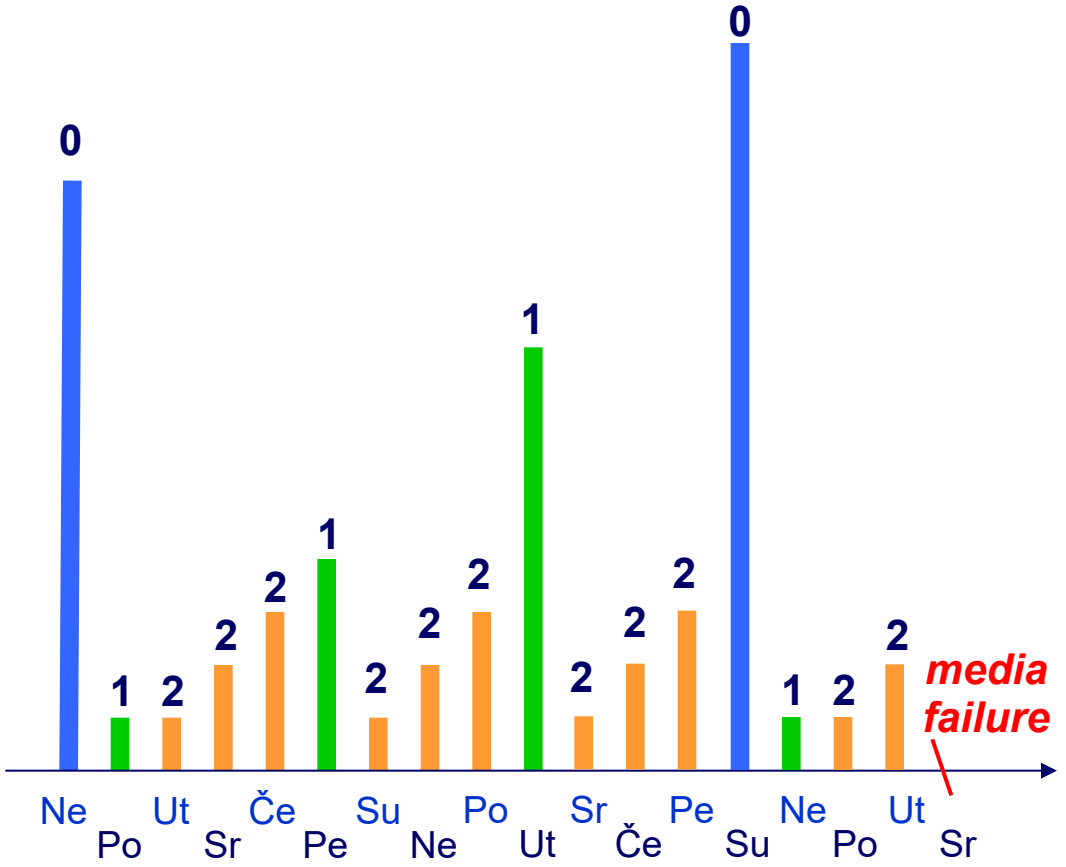
- arhiva razine 0 (*full backup*): arhiviranje **svih** blokova podataka
  - obavljati relativno rijetko
- arhiva razine *i*: arhiviranje blokova podataka koji su promijenjeni nakon izrade posljednje arhive razine *i -1*
  - obavljati relativno često

Inkrementalno arhiviranje NIJE zamjena za arhiviranje dnevnika! Inkrementalnim arhiviranjem se samo smanjuje broj zapisa dnevnika koje treba primijeniti tijekom obnove, a da se istovremeno izbjegne potreba za čestim izradama arhive razine 0.

Primjer: Inkrementalno arhiviranje baze podataka

Primjer: IBM Informix

- visinom stupca ilustrirana je veličina arhive. Boja stupca i broj na vrhu stupca označavaju razinu arhive
- slika je nacrtana s pretpostavkom visoke volatilnosti baze podataka i jednolike distribucije količine blokova koji se mijenjaju: u svakom vremenskom intervalu promijenjen je jednak broj novih blokova



- obnova nakon pogreške medija u trenutku označenom na slici moguća je pomoću:
  - zadnje arhive razine 0 + zadnje arhive razine 1 + zadnje arhive razine 2 + datoteka dnevnika nastalih nakon početka izrade zadnje arhive razine 2

Primjer: Inkrementalno arhiviranje baze podataka

- pri definiranju sheme arhiviranja u obzir treba uzeti karakterističnu periodičku raspodjelu opterećenosti sustava (ako takva postoji). Neki sustavi su npr. nedjeljom manje opterećeni u odnosu na druge dane u tjednu, te manje opterećeni noću u odnosu na druge dijelove dana. Ovdje su prikazani primjeri nekih od shema arhiviranja zasnovanih na toj pretpostavci:

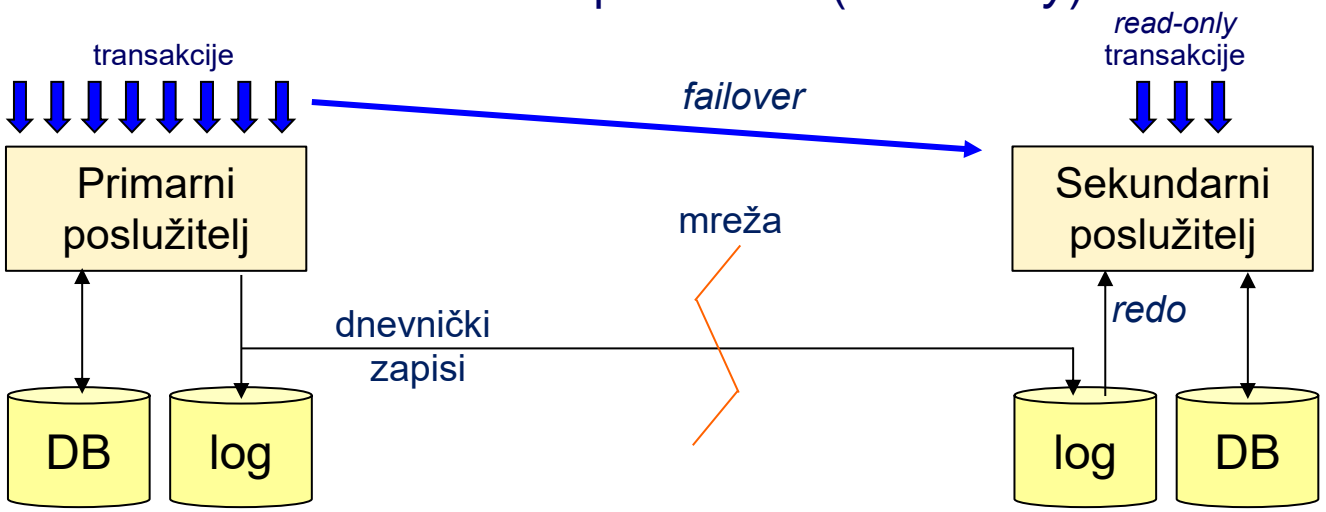
Shema 1:
- arhiva razine 0: svake četvrte nedjelje
- arhiva razine 1: svake nedjelje
- arhiva razine 2: tijekom noćnih sati svakog dana osim nedjelje
- arhiviranje datoteka dnevnika: kontinuirano

Shema 2:
- arhiva razine 0: svake nedjelje
- arhiva razine 1: svake noći osim nedjelje
- arhiviranje datoteka dnevnika: kontinuirano

Shema 3:
- arhiva razine 0: svake nedjelje
- arhiviranje datoteka dnevnika: kontinuirano

Uloga dnevnika u postizanje vrlo visoke razine raspoloživosti

- arhitektura za vrlo visoku razinu raspoloživosti (*availability*)



- transakcije se obavljaju na primarnom poslužitelju (*primary server/site*)
- dnevnički zapisi se sinkrono ili asinkrono dostavljaju i u dnevnik sekundarnog poslužitelja
- na sekundarnom poslužitelju (*secondary server/site*) se odmah ili s odgodom obavlja *redo* potvrđenih transakcija
  - sekundarni poslužitelj može se koristiti kao *read-only server*
- u slučaju kvara primarnog poslužitelja, korisničke sjednice se preusmjeravaju na dotadašnji sekundarni poslužitelj (*failover*)
- *IBM Informix: HDR (High-availability Data Replication)*
- *SQL Server: Database Mirroring; Log Shipping*

Osigurati svojstvo izdržljivosti transakcije!

- ne dopustiti gubitak efekata potvrđenih transakcija. Potrebno je:
  - spriječiti gubitak sadržaja baze podataka (kao posljedice pogreške medija)
    - arhivirati bazu podataka na medij s nezavisnim modalitetom kvarova
  - spriječiti gubitak sadržaja dnevnika (kao posljedice pogreške medija)
    - za pohranu datoteka dnevnika koristiti kvalitetnu *online* stabilnu memoriju, a arhivirati ih na medij s nezavisnim modalitetom kvarova
    - ne koristiti prevelike pojedinačne datoteke dnevnika, odnosno osigurati da se zapisi dnevnika čim prije dodatno osiguraju kopiranjem na medij s nezavisnim modalitetom kvarova (stabilnu *offline* memoriju)

- OPREZ: magnetski diskovi, trake, DVD, SSD - retencija 2-10 godina, ovisno o mikroklimatskim uvjetima u kojima se čuvaju: temperatura, vlaga, svjetlost

Literatura:

- H. Garcia-Molina, J. Ullman, J. Widom. Database system implementation. Prentice-Hall International, Inc. Upper Saddle River, New Jersey. 2000.
- R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.
- A. Silberschatz, H. Korth, S. Sudarshan. Database system concepts, 6th ed. McGraw-Hill. 2010.
- P. Bernstein, V. Hadzilacos, N. Goodman. Concurrency control and recovery in database systems. Addison-Wesley Publishing Company. Reading, Massachusetts. 1987.
- IBM Informix Backup and Restore Guide, Version 11.70, Informix Product Library, 2011.
- Oracle Database Backup and Recovery Advanced User's Guide 11g Release 2 (11.2)